

CS342: Computer Networks Lab
Assignment 02 - Application #2
**Video Streaming Simulation using TCP,
UDP, and Multithreading**

Group Number: 71

Name: PRATYUSH KUMAR SWAIN - 230101079

Name: KAVYA KUMAR AGRAWAL - 230101053

Submission Date: September 5, 2025

Contents

1	Introduction	3
2	System Architecture	3
2.1	Communication Protocol Design	3
2.2	Message Format	3
3	Performance Analysis	4
3.1	Test Environment Setup	4
3.2	Performance Metrics	4
3.3	Experimental Results	5
3.3.1	Throughput Analysis	5
3.3.2	Latency Analysis	6
3.3.3	Packet Loss Analysis (UDP)	6
3.4	Results for the comparative analysis of throughput:	6
3.4.1	TCP-	6
3.4.2	UDP-	7
3.5	Resource Utilization	7
3.5.1	CPU Usage	7
3.5.2	Memory Usage	7
4	Scheduling Policy Performance	8
4.1	Results for the comparative analysis of throughput:	8
4.1.1	FCFS-	8
4.1.2	Round-Robin-	9
5	Setup and Challenges Faced	10
5.1	Challenges Faced	10
6	Code Implementation	10
6.1	Command Line Interface	10
7	Conclusions and Observations	13
7.1	Protocol Comparison Summary	13
7.2	Key Findings	13
7.3	Recommendations	13
8	Future Enhancements	14

1 Introduction

This report presents the implementation and analysis of a video streaming simulation system using both TCP and UDP protocols. The objective is to compare the performance characteristics of these protocols under different network conditions and analyze their suitability for real-time video streaming applications.

The implementation consists of two main components:

- **Server:** A multithreaded server capable of handling multiple concurrent client connections with configurable scheduling policies (FCFS and Round-Robin)
- **Client:** A client application that can request video streaming in different resolutions and protocols

2 System Architecture

2.1 Communication Protocol Design

The video streaming simulation follows a two-phase communication protocol:

Phase 1: Connection Phase (TCP)

1. Client initiates TCP connection to server
2. Client sends Type 1 Request Message specifying video resolution
3. Server responds with Type 2 Response Message containing stream information
4. TCP connection is closed after negotiation

Phase 2: Video Streaming Phase (TCP/UDP)

1. Client chooses streaming protocol (TCP or UDP)
2. Server transmits video data using selected protocol
3. Performance metrics are collected during transmission
4. Connection is terminated after streaming completion

2.2 Message Format

All messages follow a standardized format:

Message Type	Message Length	Message Content
4 bytes (integer)	4 bytes (integer)	Variable length

Table 1: Message Format Structure

Message Types:

- Type 1: Video Request Message
- Type 2: Server Response Message
- Type 3: Video Data Message
- Type 4: Acknowledgment Message

3 Performance Analysis

3.1 Test Environment Setup

The performance tests were conducted under the following conditions:

Parameter	Value
Operating System	Windows 11
CPU	Intel Core i5
RAM	8 GB

Table 2: Test Environment Specifications

3.2 Performance Metrics

The following metrics were measured during the simulation:

Metric	Description
Throughput	Amount of video data transmitted per second (Mbps)
Latency	Time delay between request and first packet received (ms)
Packet Loss	Percentage of packets lost during UDP transmission
CPU Usage	Percentage of CPU consumed during streaming
Memory Usage	Amount of RAM used during streaming (MB)

Table 3: Performance Metrics Measured

3.3 Experimental Results

3.3.1 Throughput Analysis

```
C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 127.0.0.1 8999
TCP 1080p
[CLIENT] Connection phase response: Bandwidth=8Mbps for 1080p
=== TCP Result (1080p) ===
Latency: 0.062274 s
Elapsed: 22.171169 s
Throughput: 0.002343 Mbps
Packet Loss: 0.000000%
CPU: 0.070474%
Memory: 4820 KB
```

Figure 1: TCP Video Streaming Session

```
C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 127.0.0.1 8999
UDP 1080p
[CLIENT] Connection phase response: Bandwidth=8Mbps for 1080p
[CLIENT] Got READY signal from server, starting UDP reception (port 60209).
..
=== UDP Result (1080p) ===
Latency: 0.000676 s
Elapsed: 21.739759 s
Throughput: 0.002138 Mbps
Packet Loss: 10.500000% (received 179 / expected 200)
CPU: 0.071873%
Memory: 4844 KB
```

Figure 2: UDP Video Streaming Session

Resolution	TCP Throughput (Mbps)	UDP Throughput (Mbps)
480p	0.002249	0.002125
720p	0.002332	0.002200
1080p	0.002422	0.002294

Table 4: Throughput Comparison: TCP vs UDP

Key Observations:

- TCP throughput is more stable across different network conditions

3.3.2 Latency Analysis

Resolution	TCP Latency (ms)	UDP Latency (ms)
480p	51.974	0.573
720p	44.077	0.349
1080p	8.452	0.486

Table 5: Latency Comparison: TCP vs UDP

3.3.3 Packet Loss Analysis (UDP)

480p Loss (%)	720p Loss (%)	1080p Loss (%)
2	2.67	2

Table 6: UDP Packet Loss Under Different Network Conditions

3.4 Results for the comparative analysis of throughput:

3.4.1 TCP-

```

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 192.168.0.183 5000 TCP 480p
[CLIENT] Connection phase response: Bandwidth=3Mbps for 480p
=== TCP Result (480p) ===
Latency: 0.051974 s
Elapsed: 10.641143 s
Throughput: 0.002249 Mbps
Packet Loss: 0.000000%
CPU: 0.000000%
Memory: 4832 KB

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 192.168.0.183 5000 TCP 720p
[CLIENT] Connection phase response: Bandwidth=5Mbps for 720p
=== TCP Result (720p) ===
Latency: 0.044077 s
Elapsed: 16.093177 s
Throughput: 0.002332 Mbps
Packet Loss: 0.000000%
CPU: 0.000000%
Memory: 4824 KB

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 192.168.0.183 5000 TCP 1080p
[CLIENT] Connection phase response: Bandwidth=8Mbps for 1080p
=== TCP Result (1080p) ===
Latency: 0.008452 s
Elapsed: 21.439538 s
Throughput: 0.002422 Mbps
Packet Loss: 0.000000%
CPU: 0.072879%
Memory: 4828 KB

```

Figure 3: TCP Analysis for different video definitions.

3.4.2 UDP-

```

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>
C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 127.0.0.1 5000 UDP 480p
[CLIENT] Connection phase response: Bandwidth=3Mbps for 480p
[CLIENT] Got READY signal from server, starting UDP reception (port 57623)...
=== UDP Result (480p) ===
Latency: 0.000573 s
Elapsed: 11.043147 s
Throughput: 0.002125 Mbps
Packet Loss: 2.000000% (received 98 / expected 100)
CPU: 0.000000%
Memory: 4848 KB

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 127.0.0.1 5000 UDP 720p
[CLIENT] Connection phase response: Bandwidth=5Mbps for 720p
[CLIENT] Got READY signal from server, starting UDP reception (port 56905)...
=== UDP Result (720p) ===
Latency: 0.000349 s
Elapsed: 16.606509 s
Throughput: 0.002200 Mbps
Packet Loss: 2.666667% (received 146 / expected 150)
CPU: 0.188179%
Memory: 4852 KB

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>client 127.0.0.1 5000 UDP 1080p
[CLIENT] Connection phase response: Bandwidth=8Mbps for 1080p
[CLIENT] Got READY signal from server, starting UDP reception (port 63841)...
=== UDP Result (1080p) ===
Latency: 0.000406 s
Elapsed: 22.190385 s
Throughput: 0.002204 Mbps
Packet Loss: 2.000000% (received 196 / expected 200)
CPU: 0.070413%
Memory: 4844 KB

C:\Users\Kavya Agrawal\source\repos\client\x64\Debug>

```

Figure 4: UDP Analysis for different video definitions.

3.5 Resource Utilization

3.5.1 CPU Usage

Resolution	TCP CPU Usage (%)	UDP CPU Usage (%)
480p	0.00023	0.00018
720p	0.00027	0.18818
1080p	0.07288	0.07041

Table 7: CPU Usage Comparison

3.5.2 Memory Usage

Resolution	TCP Memory (KB)	UDP Memory (KB)
480p	4832	4848
720p	4824	4852
1080p	4828	4844

Table 8: Memory Usage Comparison

4 Scheduling Policy Performance

4.1 Results for the comparative analysis of throughput:

4.1.1 FCFS-

```
C:\Users\Kavya Agrawal\source\repos\server\x64\Debug>server 8999 FCFS
[SERVER] Listening on 8999, Policy=FCFS
[SERVER] Connection phase done for 1080p
[SERVER] Enqueued client (TCP 1080p, 200 packets, size=1500)
[SERVER] FCFS: serving client (TCP 1080p)
[SERVER] Connection phase done for 1080p
[SERVER] Enqueued client (UDP 1080p, 200 packets, size=1500)
[SERVER] FCFS: finished client (TCP 1080p)
[SERVER] FCFS: serving client (UDP 1080p)
[SERVER] Drop UDP packet #4 (1080p)
[SERVER] Drop UDP packet #11 (1080p)
[SERVER] Drop UDP packet #22 (1080p)
[SERVER] Drop UDP packet #23 (1080p)
[SERVER] Drop UDP packet #41 (1080p)
[SERVER] Drop UDP packet #66 (1080p)
[SERVER] Drop UDP packet #72 (1080p)
[SERVER] Drop UDP packet #77 (1080p)
[SERVER] Drop UDP packet #78 (1080p)
[SERVER] Drop UDP packet #90 (1080p)
[SERVER] Drop UDP packet #105 (1080p)
[SERVER] Drop UDP packet #108 (1080p)
[SERVER] Drop UDP packet #137 (1080p)
[SERVER] Drop UDP packet #139 (1080p)
[SERVER] Drop UDP packet #146 (1080p)
[SERVER] Drop UDP packet #147 (1080p)
[SERVER] Drop UDP packet #153 (1080p)
[SERVER] Drop UDP packet #157 (1080p)
[SERVER] Drop UDP packet #169 (1080p)
[SERVER] Drop UDP packet #187 (1080p)
[SERVER] Drop UDP packet #195 (1080p)
[SERVER] FCFS: finished client (UDP 1080p)
```

Figure 5: FCFS for multi-threading.

4.1.2 Round-Robin-

```
C:\Users\Kavya Agrawal\source\repos\server\x64\Debug>server 8999 RR
[SERVER] Listening on 8999, Policy=RR
[SERVER] Connection phase done for 1080p
[SERVER] Enqueued client (UDP 1080p, 200 packets, size=1500)
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] Drop UDP packet #4 (1080p)
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] Connection phase done for 1080p
[SERVER] Enqueued client (TCP 1080p, 200 packets, size=1500)
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] Drop UDP packet #78 (1080p)
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
[SERVER] RR: client (UDP 1080p), sending 5 packets
[SERVER] RR: client (TCP 1080p), sending 5 packets
```

Figure 6: RR for multi-threading.

Analysis:

- Round-Robin provides better fairness among clients
- FCFS shows lower complexity but higher response time variance
- Round-Robin prevents client starvation effectively

5 Setup and Challenges Faced

5.1 Challenges Faced

Network Configuration Issues:

- Bridged adapter caused IP conflicts in shared network
- Solution: Used host-only network for consistent testing environment

Threading Synchronization:

- Race conditions in request queue management
- Inconsistent performance measurements with multiple threads
- Solution: Implemented proper mutex locks and atomic operations

6 Code Implementation

6.1 Command Line Interface

The implementation supports the required command-line interface:

mdframed

Server:

```
./server <Server Port> <Scheduling Policy: FCFS/RR>
```

```
Example: ./server 8999 RR
```

Client:

```
./client <Server IP> <Server Port> <Mode: TCP/UDP> <Resolution:  
480p/720p/1080p>
```

```
Example: ./client 192.168.1.100 8999 UDP 1080p
```

```

// Initialize Winsock
WSADATA ws;
if (WSAStartup(MAKEWORD(2, 2), &ws) != 0) {
    fprintf(stderr, "WSAStartup failed.\n");
    return EXIT_FAILURE;
}

// --- First contact with server ---
SOCKET ctrl_sock = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in srv_addr;
ZeroMemory(&srv_addr, sizeof(srv_addr));
srv_addr.sin_family = AF_INET;
srv_addr.sin_port = htons(srv_port);
InetPtonA(AF_INET, srv_ip, &srv_addr.sin_addr);

connect(ctrl_sock, (struct sockaddr*)&srv_addr, sizeof(srv_addr));

// Send handshake message
struct Message handshake, reply;
handshake.type = 1;
handshake.length = (int)strlen(quality);
strcpy_s(handshake.data, sizeof(handshake.data), quality);

send(ctrl_sock, (char*)&handshake, sizeof(handshake), 0);
recv(ctrl_sock, (char*)&reply, sizeof(reply), 0);

reply.data[reply.length] = '\0';
printf("[CLIENT] Handshake response: %s\n", reply.data);

closesocket(ctrl_sock);

int pkt_expected = expected_packets_for_res(quality);

```

Figure 7: Initialisation of Winsock and handshaking message.

```

// --- TCP Streaming Mode ---
if (_stricmp(transport, "TCP") == 0) {
    SOCKET tcp_sock = socket(AF_INET, SOCK_STREAM, 0);
    connect(tcp_sock, (struct sockaddr*)&srv_addr, sizeof(srv_addr));

    struct Message req;
    req.type = 3;
    sprintf_s(req.data, "%s %s %d", transport, quality, 0);
    req.length = (int)strlen(req.data);

    double t0 = now_seconds();
    send(tcp_sock, (char*)&req, sizeof(req), 0);

    receive_tcp(tcp_sock, pkt_expected, quality, t0);

    closesocket(tcp_sock);
}

```

Figure 8: Tcp streaming mode.

```

// --- UDP Streaming Mode ---
else {
    SOCKET udp_sock = socket(AF_INET, SOCK_DGRAM, 0);

    struct sockaddr_in local;
    ZeroMemory(&local, sizeof(local));
    local.sin_family = AF_INET;
    local.sin_addr.s_addr = INADDR_ANY;
    local.sin_port = 0; // let OS assign ephemeral port
    bind(udp_sock, (struct sockaddr*)&local, sizeof(local));

    // Retrieve OS-assigned port
    struct sockaddr_in bound_addr;
    int blen = sizeof(bound_addr);
    getsockname(udp_sock, (struct sockaddr*)&bound_addr, &blen);
    int udp_port = ntohs(bound_addr.sin_port);

    // Inform server about UDP port
    SOCKET notify_sock = socket(AF_INET, SOCK_STREAM, 0);
    connect(notify_sock, (struct sockaddr*)&srv_addr, sizeof(srv_addr));

    struct Message req;
    req.type = 3;
    sprintf_s(req.data, "%s %s %d", transport, quality, udp_port);
    req.length = (int)strlen(req.data);
    send(notify_sock, (char*)&req, sizeof(req), 0);

    // Wait for READY signal
    struct Message ready_msg;
    if (recv(notify_sock, (char*)&ready_msg, sizeof(ready_msg), 0) > 0 &&
        ready_msg.type == 5) {
        printf("[CLIENT] Server READY. Listening on UDP port %d...\n", udp_port);
    }
    else {
        fprintf(stderr, "[CLIENT] READY signal not received.\n");
        closesocket(udp_sock);
        closesocket(notify_sock);
        WSACleanup();
        return EXIT_FAILURE;
    }

    double t0 = now_seconds();
    receive_udp(udp_sock, pkt_expected, quality, t0);

    closesocket(udp_sock);
    closesocket(notify_sock);
}

WSACleanup();
return EXIT_SUCCESS;

```

Figure 9: Udp streaming mode.

7 Conclusions and Observations

7.1 Protocol Comparison Summary

Aspect	TCP	UDP
Reliability	High (guaranteed delivery)	Low (best effort)
Speed	Moderate (protocol overhead)	High (minimal overhead)
Resource Usage	Higher (connection state)	Lower (stateless)
Suitable For	Quality-critical streaming	Real-time streaming

Table 9: TCP vs UDP Protocol Comparison

7.2 Key Findings

1. **Performance Trade-offs:** UDP offers lower latency but at the cost of reliability
2. **Scalability:** Round-Robin scheduling provides better fairness and prevents client starvation
3. **Network Sensitivity:** TCP performance degrades more gracefully under network congestion

7.3 Recommendations

Based on the experimental results:

- **Live Streaming:** Use UDP for real-time applications where low latency is critical
- **Video-on-Demand:** Use TCP for applications requiring guaranteed data delivery
- **Server Design:** Implement Round-Robin scheduling for multi-client scenarios
- **Adaptive Streaming:** Consider hybrid approaches that switch protocols based on network conditions

8 Future Enhancements

In order to further improve the performance, scalability, and reliability of the video streaming system, the following enhancements can be considered for future development:

1. **Implementation of Adaptive Bitrate Streaming (ABR):** Introduce adaptive bitrate mechanisms to dynamically adjust the video quality based on available network bandwidth and device capabilities. This ensures smooth playback even under fluctuating network conditions.
2. **Addition of Video Compression Simulation:** Implementing video compression algorithms (e.g., H.264, HEVC) in the simulation would provide insights into real-world performance. This enhancement would enable a closer approximation of actual streaming behavior under constrained bandwidth.
3. **Real-Time Network Condition Monitoring:** Introduce tools to continuously monitor parameters such as latency, jitter, throughput, and packet loss. Such feedback could be used to trigger adaptive strategies, improve diagnostics, and optimize streaming performance.
4. **Support for Multicast Streaming:** Extend the system to support multicast-based delivery, reducing redundant transmissions when multiple clients request the same video stream. This would significantly enhance scalability for large-scale deployments.
5. **Enhanced Security Features:** Implement encryption (e.g., TLS/DTLS) and authentication mechanisms to secure video streams and prevent unauthorized access or tampering.